

MÓDULO III

Introdução à Programação II

MÓDULO III

O que é HTML?

Linguagem base dos websites

Camadas de desenvolvimento

Existem 3 linguagens básicas que utilizamos para criar websites: **HTML**, **CSS** e **JavaScript**.

O **HTML** é a linguagem que irá exibir a informação. O **CSS** é a linguagem que vai deixar essa informação bonita. O **JavaScript** é a linguagem que vai fazer essa informação receber alguns comportamentos, como por exemplo ao criar um submenu ou controlar algo que aparece e desaparece na tela.

O **HTML** sem dúvida é a mais importante de todas, por que como dissemos no começo, é ela que exibe a informação. Além de exibir a informação, ela dá significado. Isso é importante por que alguns sistemas como o Google, que irão ler sua página, precisam entender o que é cada elemento nela e o que cada um desses elementos significam.

MÓDULO III

O nome HTML

O acrônimo **HTML** significa em inglês: **HyperText Markup Language**. Para gente aqui fica: **Linguagem de Marcação de Hipertexto**. Bonito, né?

Por trás das palavras **Hipertexto** e **Marcação** tem muita história e guardam a real essência da função do **HTML**. Você vai saber daqui a pouco, onde falamos sobre Semântica, que nada mais é do que a organização da informação usando **HTML**.

Se você tiver que guardar alguma coisa sobre o que é **HTML**, guarde isso: ***HTML** serve para dar significado e organizar a informação dos websites.*

Tendo isso em mente, você já saberá muito mais do que a maioria dos profissionais por aí

MÓDULO III

Marcação

Já que o **HTML** serve para dar significado para a informação, como ele faz isso? Simples: ele marca a informação com as tags.

Por exemplo, para falarmos que um título é um título colocamos um pedaço de texto entre uma tag chamada **H1**. Veja o código abaixo:

```
1. <h1>Aqui vai o texto que é um título</h1>
```

E dessa forma vamos fazendo todos os outros elementos. Um parágrafo, por exemplo:

```
1. <p>Aqui vai muito texto, um parágrafo</p>
```

O resultado fica assim:

Aqui vai o texto que é um título

Aqui vai muito texto, um parágrafo

MÓDULO III

O que é Semântica?

Organizando a informação: forma linear ou não linear

Você já pensou em como se organiza informação? A informação - um texto por exemplo - pode ser organizada de forma **linear** ou **não linear**. Imagine um livro. Toda a informação contida no livro está organizada de forma linear, isto é, há começo, meio e fim. Você não consegue absorver todo o significado da história contada se iniciar sua leitura pela metade do livro. Você precisa, necessariamente, iniciar do primeiro capítulo até o último para absorver o máximo possível de informação sobre a história. As enciclopédias tem suas informações organizadas de forma **não linear**. Isto é: as informações não estão organizadas em uma ordem específica, mas sim de forma relacional, associativa. Há uma série de assuntos e todos eles estão organizadas de uma maneira que se relacionam por meio de referências. Por exemplo: quando você procura informações sobre carros na enciclopédia, ao final do texto você encontra uma série de outras referências aos assuntos relacionados, como por exemplo: motores a combustão interna, rodas, combustíveis e etc.

MÓDULO III

Como você e a web funcionam

É mais ou menos assim que a sua mente funciona também: quando você pensa em um assunto, você consegue fazer uma série de associações com outros assuntos, formando uma ideia, uma memória, uma lembrança. Por isso seu cérebro consegue guardar essas informações de uma jeito que se torna fácil de recuperá-las não apenas pensando nessa informação diretamente, mas também quando se pensa nos assuntos relacionados a esta informação.

Se levarmos isso para web, percebemos que apenas uma tag faz esse trabalho de referência e associação de informação: o **link**. Na web você relaciona informações, sites e etc utilizando diretamente a tag "A". Quando você linka um texto, uma imagem, um gráfico etc, você associa essa informação ao destino do **link**. Você está referenciando assuntos, como na enciclopédia. Sem os **links** não há web. Não conseguimos associar informação alguma. Perde-se a referência. Nós usamos o **link** todos os dias e aposto que a metade dos desenvolvedores não imagina a importância dessa tag.

MÓDULO III

O que são Tags, Elementos e Atributos? Conhecendo as principais Tags, Elementos e seus Atributos

O que são Tags?

Tags são o conjunto de caracteres que formam um elemento, ou seja, quando nos referenciamos à Tag "**p**" estamos falando disso: **<p>**

Existem dois tipos de Tags, as que necessitam de fechamento e as que não necessitam de fechamento.

Para as que necessitam de fechamento, utilizamos o sinal de menor (<), seguido do nome do elemento e o sinal de maior (>) para abertura. Para fechamento, utilizamos o sinal de menor (<), seguido de barra (/), nome do elemento e o sinal de maior (>).

```
1. <!-- Exemplo de elemento que necessita de fechamento -->
2. <p>A tag do elemento de parágrafo necessita de fechamento.</p>
```

MÓDULO III

Os elementos que não necessitam de fechamento, também conhecidos como ***elementos vazios***, somente utilizamos o sinal de menor (<), seguido do nome do elemento e o sinal de maior (>).

```
1. <!-- Exemplo de elemento vazio -->  
2. Texto utilizando<br>  
3. quebra de linha
```


MÓDULO III

O que são Elementos?

Elementos são formados a partir de **Tags** e entre as **Tags** é que está o conteúdo do Elemento. Se quisermos exibir um parágrafo em um site utilizamos a **Tag** `<p>` que semanticamente quer dizer que o conteúdo que se espera nesse Elemento é um parágrafo.

Alguns exemplos:

```
1. <!-- A Tag <label> define que o conteúdo do Elemento é uma label (rótulo) -->
2. <label>Informe o seu nome</label>
```

```
<!-- A Tag <address> define que o conteúdo do Elemento é um endereço (endereço físicos à virtuais) -->
<address>
  Este guia é uma iniciativa da comunidade e do Tableless <a href="http://tableless.com.br/contato">www.tableless.com.br</a>
</address>
```

MÓDULO III

Não confunda Tags com Elementos

As **Tags** servem para marcar onde começam e terminam os **Elementos**. Já os **Elementos** são uma parte conceitual/semântica que têm um começo e fim determinados.

Parece uma diferença boba, mas mantenha ela sempre em mente e isso vai fazer toda a diferença no seu entendimento de **HTML**.

O que são Atributos?

Atributos são informações que passamos na **Tag** para que ela se comporte da maneira esperada. Existem **atributos** globais (que funcionam em todas as **Tags**) e específicos (que são direcionados para cada **Tag**, através de especificação).

Os **Atributos** possuem nome e um valor, existem **Atributos** que você vai usar praticamente sempre e existem outros que serão mais raros.

MÓDULO III

Atributos globais

accesskey

Determina uma (ou mais) tecla(s) de atalho para o elemento.

Para definir mais de uma tecla, coloque-as separadas por espaço.

draggable

Define se um elemento é arrastável ou não.

class

Determina uma (ou mais) classe(s) para o elemento. Para definir mais de uma classe, coloque-as separadas por espaço.

hidden

Oculto o elemento onde for declarado.

MÓDULO III

id

É o identificador único do elemento. Somente deve ser declarado um id por elemento.

E este id não deve ser repetido na mesma página.

lang

Determina o idioma em que está escrito o conteúdo do elemento.

style

Determina propriedades CSS diretamente no elemento.

tabindex

Organiza os elementos por ordem de tabulação. Deve-se usar valor numérico.

title

Representa um auxílio ao elemento. Semelhante a dica do elemento.

MÓDULO III

Como você pôde ver, a sintaxe de **Atributos** é muito simples, porém, por mais simples que seja é sempre bom ter em mente boas práticas para que se tenha qualidade e fácil manutenção em seu **HTML**. Existem **Atributos** em que os valores não precisam ser passados entre **aspas**, mas manter um padrão ajuda você e ajuda quem quer que seja que no futuro tenha que trabalhar com o seu **HTML**. Portanto, não é uma regra mas uma boa prática você envolver os valores dos **Atributos** entre **aspas**.

```
1.      <!-- Isso funciona, mas não é recomendável -->
2.      <a href="http://tableless.com.br" target=_blank>www.tableless.com.br</a>
3.
4.      <!-- Isso funciona e é recomendável -->
5.      <a href="http://tableless.com.br" target="_blank">www.tableless.com.br</a>
```

MÓDULO III

Na prática

Agora que você entendeu separadamente o papel das **Tags**, **Elementos** e **Atributos**, vamos ver um exemplo prático:

```
<!-- A Tag <img> define que o conteúdo do Elemento é uma imagem e os atributos que utilizamos são src e alt -->  

```

Analizando o exemplo:

No exemplo acima definimos o caminho onde está a imagem com o **Atributo *src*** e utilizamos o **Atributo *alt*** para descrever que imagem é essa (*a utilização do atributo **alt** é altamente recomendado, pois, mesmo que a imagem não seja carregada por qualquer motivo, o usuário conseguirá identificar que ali era para ser carregado o logo do HTML5).*

MÓDULO III

Estrutura básica

Iniciando o código básico de **HTML**

O documento **HTML** sempre inicia com o que chamamos de **estrutura básica**. Esta estrutura é quase que imutável. Sempre será dessa forma e você sempre, sempre começará seu **HTML** começando por esse código. Geralmente os editores como o **VS Code** já tem atalhos para iniciar os documentos **HTMLs** com essa estrutura, logo, você não precisa se preocupar em decorá-la, mas é bom que faça. Veja abaixo como ela se inicia:

```
1. <!DOCTYPE html>
2. <html lang="pt-br">
3.   <head>
4.     <title>Título da página</title>
5.     <meta charset="utf-8">
6.   </head>
7.   <body>
8.     Aqui vai o código HTML que fará seu site aparecer.
9.   </body>
10. </html>
```

MÓDULO III

É possível compreender o documento em **HTML** de uma maneira muito simples, através de uma divisão de blocos das tags essenciais, conforme a seguinte estrutura:

- Definição do documento (**doctype**)
- Cabeça (**head**)
- Corpo (**body**)

Doctype - Definindo o documento

Uma coisa importante: **SEMPRE** deve existir o **doctype**, que é este código `<!DOCTYPE html>`.

O **doctype** não é uma tag **HTML**, mas uma instrução para o navegador e outros programas que podem ler seu site, que o código encontrado ali é um código **HTML**. Assim eles sabem o que fazer para mostrar seu site da melhor forma possível. Lembre-se: o **doctype** é **OBRIGATÓRIO** e deve ser sempre a **PRIMEIRA LINHA** do seu documento.

MÓDULO III

HEAD

Contém informações que não são transpostas visivelmente para o **usuário/leitor** do documento.

São dados implícitos, de uso e controle do documento: vinculação com outros arquivos, aplicação de lógica de programação de **scripts** e metadados. Na prática, todo o conteúdo do cabeçalho fica delimitado entre a abertura e fechamento tag **head**.

MÓDULO III

BODY

Trata-se do documento em si, ou seja, a informação legível para o **usuário/leitor** do documento. É todo e qualquer texto que se deseja apresentar, assim como toda e qualquer forma de mídia de saída (**imagens, sons, miniaplicativos embutidos, conteúdo multimídia, etc**). Além disso, toda a apresentação de entrada de dados (formulários) também se aplica nesta seção do documento.

Na prática, o corpo do documento é delimitado pelo par de **tags <body> e </body>**.

Este é o preceito básico que deve estar muito bem claro para você: onde as marcações se aplicam, e quais são os resultados deste modelo. Por exemplo: se vocês deseja informar conteúdo textual para saída legível ao usuário do seu sistema web, esta marcação deverá obrigatoriamente estar no bloco do corpo da página. Ainda: para definir qual o tipo de codificação da página (a metatag **<meta>** deve constar na informação do documento), esta deve obrigatoriamente estar marcada no cabeçalho do mesmo documento.

Dentro do elemento **BODY** sua estrutura de página terá os elementos semânticos da construção da sua página, onde serão declarados e identificados **cabeçalhos, rodapé, conteúdo principal**, etc.

MÓDULO III

O que são metatags? Informação sobre seu site

A **Meta Tag**, representada pela tag <meta> é uma tag diferenciada das demais por não ter nenhum efeito aparente na página em si, mas sim por ser responsável por ações externas à página, como por exemplo informar à buscadores como Google ou Bing algumas informações a respeito da página, como título e uma breve descrição.

Tipos de Meta Tags

Keywords

Aqui é um dos locais onde os motores de busca procuram informações a respeito da página.

Copyright

Direito autoral da página.

Author

O nome do autor da página.

Description

Descrição da página.

Expires

Data em que o documento deve ser considerado expirado.

MÓDULO III

O que é CSS?

O Cascading Style Sheets (**CSS**) é uma linguagem utilizada para definir a apresentação (aparência) de documentos que adotam para o seu desenvolvimento linguagens de marcação (como **XML**, **HTML** e **XHTML** e etc..). O **CSS** define como serão exibidos os elementos contidos no código de um documento e sua maior vantagem é efetuar a separação entre o formato e o conteúdo de um documento.

Por que o CSS foi criado?

Com a evolução dos recursos de programação, as tecnologias estavam adotando cada vez mais estilos e variações para deixá-las mais elegantes e atrativas para os usuários. Com isto, linguagens de marcação simples como o **HTML**, que era destinada para apresentar os conteúdos, também precisaram ser aprimoradas

Foram criadas novas tags e atributos de estilo para o **HTML** e em resumo, ele passou a exercer tanto a função de estruturar o conteúdo quanto de apresentá-lo para o usuário final. Entretanto, isto começou a trazer um problema para os desenvolvedores, pois não havia uma forma de definir, por exemplo, um padrão para todos os cabeçalhos ou conteúdos em diversas páginas. Ou seja, as alterações teriam que ser feitas manualmente, uma a uma.

MÓDULO III

A partir destas complicações, nasceu o **CSS**. Primariamente, foi desenvolvido para habilitar a separação do conteúdo e formato de um documento (na linguagem de formatação utilizada) de sua apresentação, incluindo elementos como **cores, formatos de fontes e layout**. Esta separação proporcionou uma maior flexibilidade e controle na especificação de como as características serão exibidas, permitiu um compartilhamento de formato e reduziu a repetição no conteúdo estrutural de um documento.

Com isto, as linguagens de marcação passaram novamente a exercer sua função de marcar e estruturar o conteúdo de um documento, enquanto o **CSS** encarregou-se da aplicação dos estilos necessários para a aparência dela. Isto é feito por meio da criação de um arquivo externo que contém todas as regras aplicadas e com isto, é possível fazer alterações de estilo em todas as páginas de um site e outros documentos que utilizam **CSS** de forma fácil e rápida.

O **CSS** também permite que as mesmas marcações de um documento sejam apresentadas em diferentes estilos, conforme os métodos de renderização (*como em uma tela, impressão, via voz, baseadas em dispositivos táteis, etc.*). A maioria dos menus em cascata, estilos de cabeçalho e rodapé de páginas da internet, por exemplo, atualmente são desenvolvidos em **CSS**.

MÓDULO III

Como escrever código CSS

Sintaxe

Para escrever código **CSS** é necessário seguir uma regra. A regra é uma declaração que possui uma sintaxe própria bem simples que define a forma com que o estilo será aplicado aos nossos elementos **HTML**.

Você pode ver a regra a seguir:

```
1.  seletor{  
2.    propriedade: valor;  
3.  }
```

A nossa regra como pode ser visto é composta de três partes principais: **um seletor, uma propriedade e um valor** a se aplicar.

Explicando de forma grotesca o *seletor* nada mais é do que o nosso elemento **HTML** ao qual queremos aplicar a regra (por exemplo: **div, body**).

A *propriedade* é o atributo do elemento que será aplicado a regra. (por exemplo: **color, font, position**).

Valor é a característica que o elemento irá assumir (por exemplo: **cor azul, tamanho 14 para a fonte**).

MÓDULO III

Exemplificando

Vamos para um exemplo:

```
1. body{  
2.     background-color: #000;  
3. }
```

No nosso exemplo o elemento ao qual estamos aplicando a regra é o **BODY** este é o nosso **seletor**, a nossa **propriedade** é *background-color* que define a cor de fundo do documento e o nosso **valor** *#000* que é igual a cor preta.

MÓDULO III

Incluindo CSS na página

Existem três formas para incluir o código **CSS** em seu projeto

Inline

A primeira forma de aplicar **CSS** a uma página é utilizando o atributo **style** em elementos do **HTML**:

```
1. <p style="color: blue">Parágrafo com fonte azul.</p>
2. <p>Esse outro parágrafo não é azul, a não ser que
3. exista <span style="color: red">CSS em outro lugar</span>.</p>
```

Interno

A segunda forma é utilizar a tag **style** dentro do **head** da página **HTML**:

```
1. <head>
2.   <style type="text/css">
3.     seletor { propriedade: valor; }
4.   </style>
5. </head>
```


MÓDULO III

Externo

E a última - porém a mais utilizada - maneira de aplicar **CSS** é criar um ou mais arquivos com extensão **.css** e incluí-los na estrutura **head** do **HTML**:

```
1. <head>
2.   <link rel="stylesheet" type="text/css" href="reset.css">
3.   <link rel="stylesheet" type="text/css" href="styles.css">
4. </head>
```

MÓDULO III

Qual a diferença entre CLASS e ID?

Em **HTML** e **CSS**, há a possibilidade de aplicar estilos através de '*class*' e '*id*' e, em **JavaScript** é possível identificar algum elemento de uma página por sua classe, *id* ou *tag*. Mas qual a diferença entre '*class*' e '*id*'?

O que são classes?

As **classes** são uma forma de identificar um grupo de elementos.
Através delas, pode-se atribuir formatação a VÁRIOS elementos de uma vez. Exemplo:

Código CSS:

```
1. .class1 {  
2.     background: blue;  
3. }
```

MÓDULO III

Código HTML:

```
1. <!DOCTYPE html>
2. <html lang="pt-br">
3.   <head>
4.     <title></title>
5.     <meta charset="utf-8">
6.   </head>
7.   <body>
8.     <div class="classe1">Div1</div>
9.     <div class="classe1">Div2</div>
10.    <div class="classe1">Div3</div>
11.    <div class="classe1">Div4</div>
12.    <div class="classe1">Div5</div>
13.  </body>
14. </html>
```

Então, todas as '**divs**' que estiverem com a classe "**classe1**" estarão com um background **azul**(blue).

MÓDULO III

O que são Ids?

As **ids** são uma forma de identificar um elemento, e devem ser ÚNICAS para cada elemento.

É como se fossem impressões digitais de nossos dedos ou RGs.

Através delas, pode-se atribuir formatação a um elemento em especial. Exemplo:

Código CSS:

```
1. #idUm {  
2.     background: blue;  
3. }  
4. #idDois {  
5.     background: yellow;  
6. }  
7. #idTres {  
8.     background: lightblue;  
9. }  
10. #idQuatro {  
11.     background: lightgreen;  
12. }
```

MÓDULO III

Código HTML:

```
1. <!DOCTYPE html>
2. <html lang="pt-br">
3.   <head>
4.     <title></title>
5.     <meta charset="utf-8">
6.   </head>
7.   <body>
8.     <div id="idUm">Div1</div>
9.     <div id="idDois">Div2</div>
10.    <div id="idTres">Div3</div>
11.    <div id="idQuatro">Div4</div>
12.  </body>
13. </html>
```

Então, como mostra o código acima, não podemos repetir uma **'id'**.
Deve ser especialmente única para cada elemento.

MÓDULO III

Selecionando elementos

Entenda como formatar os elementos do CSS

Seletores CSS ajudam a especificar melhor o elemento que deve ser afetado.

Vejamos alguns seletores:

Elemento Filho (A > B)

Para selecionar um elemento (B) que tenha um elemento pai específico (A), deve-se utilizar o ">".

```
> nav.menu-principal > ul {  
  padding: 0;  
}  
// neste caso, somente o elemento ul, que tem como pai um elemento nav com classe "menu-principal", receberá a alteração de preenchimento
```

MÓDULO III

Elemento Irmão (A + B)

Quando um elemento (B) está ao lado de outro específico (A).

```
1. label + input {  
2.   border: 1px solid #333;  
3. }  
4. // todos os inputs que estiverem ao lado de um label receberão borda cinza
```

Não este elemento (A:not(b))

Exclui determinado elemento (A) de acordo com a especificação (B).

```
1. div:not(.principal) {  
2.   border: 1px solid #F00;  
3. }  
4. // todas as divs que não tenham a classe "principal" receberão borda vermelha
```

MÓDULO III

Primeiro Filho (A:first-child)

Seleciona o primeiro elemento que aparecer, de acordo com o elemento definido (A).

```
1. li:first-child {  
2.     background-color: #0F0;  
3. }  
4. // O primeiro li terá fundo verde
```

Enésimo Elemento (A:nth-child(b))

Seleciona o elemento definido (A) de acordo com a posição informada (b).

```
1. li:nth-child(7) {  
2.     background-color: #666;  
3. }  
4. // O sétimo li terá fundo cinza
```


MÓDULO III

Seletores complexos

Selecionando diferentes elementos de um formulário

Tipos de *Inputs*:

```
1. input[type="text"] { ... }  
2. input[type="email"] { ... }
```

No exemplo acima, selecionamos respectivamente elementos **input** do tipo *text* e *email*.

Input checkbox

```
1. input[type="checkbox"]:checked { ... }
```

A própria declaração é bem intuitiva. Seleciona elementos do tipo *checkbox* que estiverem selecionado.

MÓDULO III

Selecionando elementos pela negação

```
1. div:not(.estilo) { ... }
```

Acima, selecionamos todas as **divs** do documento, *exceto* as que possuem a classe **.estilo**.

Input inativo

```
1. input:disabled { ... }
```

Acima, selecionamos um input que esteja com o atributo **disabled**, estando assim inativo.

MÓDULO III

Box Model

Entendendo como funciona elementos no HTML

Sempre quando vamos estilizar algum elemento via **CSS**, é comum, e precisamos estar cientes, que alguma alteração que iremos fazer possa impactar em outros elementos.

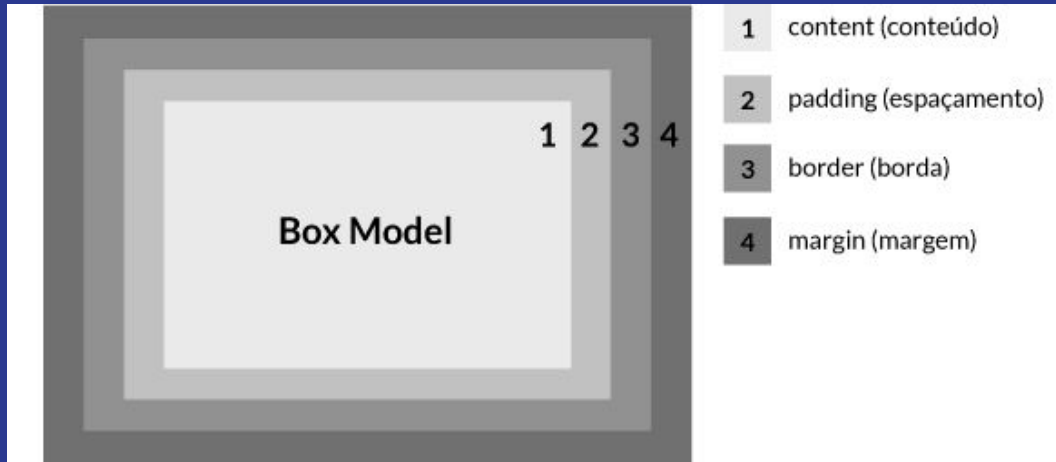
Isso fica fácil de compreender quando entendemos o conceito de **box model**.

Basicamente, a idéia do **box model** é composta por quatro partes:

- *conteúdo*
- *espaçamento*
- *bordas*
- *margens*

Resumindo, podemos dizer que o **box model** trata-se de como as 4 propriedades acima se relacionam para compor a dimensão do elemento.

MÓDULO III



```
1. .classe {  
2.     width: 50px;  
3.     height: 50px;  
4.     border: 1px solid gray;  
5.     padding: 10px 20px;  
6. }
```

MÓDULO III

Se aplicarmos a classe acima em um elemento, ele vai ter as dimensões que setamos (**50 pixels de altura e largura**), certo? Errado. O elemento vai ser renderizado com **72 pixels de altura e 92 pixels de largura**. Isso acontece devido ao fato das propriedades *padding* e *border* serem somadas à largura e altura já definidas, aumentando as dimensões do elemento. Isso nos leva a ver que as propriedades *width* e *height* definem as dimensões do seu conteúdo e não do elemento como um todo.

Largura

50 (largura definida) +
20 (padding left) +
20 (padding right) +
1 (border left) +
1 (border right) => **92** pixels de largura

Altura

50 (altura definida) +
10 (padding top) +
10 (padding bottom) +
1 (border top) +
1 (border bottom) => **72** pixels de altura

MÓDULO III

Um exemplo prático para vermos a dor de cabeça que você pode ter no seu dia a dia.

Imagine que você precise ter um elemento que ocupa **100%** da largura disponível. Mas também precisa que esse elemento tenha **10 pixels** de *padding* e uma borda de **1 pixel**.

```
1. .dor-de-cabeca {  
2.     width: 100%;  
3.     padding: 10px;  
4.     border: solid 1px gray;  
5. }
```

Seu elemento com a classe **dor-de-cabeca** ultrapassa o limite de **100%** que você tinha definido, provavelmente *quebrando* o layout. A nova largura dele é resultante da soma: **100%** (*largura definida*) + **20 pixels** (*padding left e right*) + **2 pixels** (*border left e right*).

MÓDULO III

A propriedade box-sizing

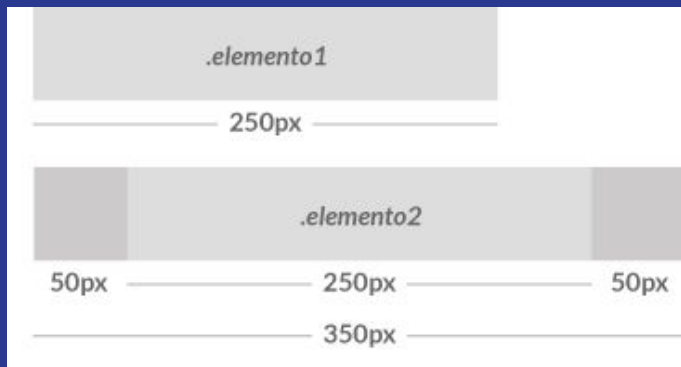
Já vimos o tanto de facilidades que o **CSS3** trouxe para a vida dos desenvolvedores. Uma delas é a propriedade **box-sizing** que permite que mudemos o comportamento do **box-model**. Por padrão, todos os elementos possuem o valor **content-box** para essa propriedade. Trata-se do que vimos nos exemplos acima: o **box-model** é definido apenas pelo conteúdo

```
1. box-sizing: content-box;
```

Um outro exemplo para exemplificar o box-model com o uso do **content-box**. Considere os elementos, a seguir:

MÓDULO III

Os elementos são definidos com a mesma largura, mas como podemos ver na imagem abaixo, o **elemento2** é renderizado maior devido ao **padding**.



```
1.  .elemento1 {  
2.      background: #ddd;  
3.      width: 250px;  
4.      height: 50px;  
5.  }  
6.  
7.  .elemento2 {  
8.      background: #ddd;  
9.      width: 250px;  
10.     height: 50px;  
11.     padding: 0 50px;  
12. }
```


MÓDULO III

A solução border-box

A propriedade **box-sizing** permite um novo valor que resolve todos os problemas apresentados nos exemplos anteriores:

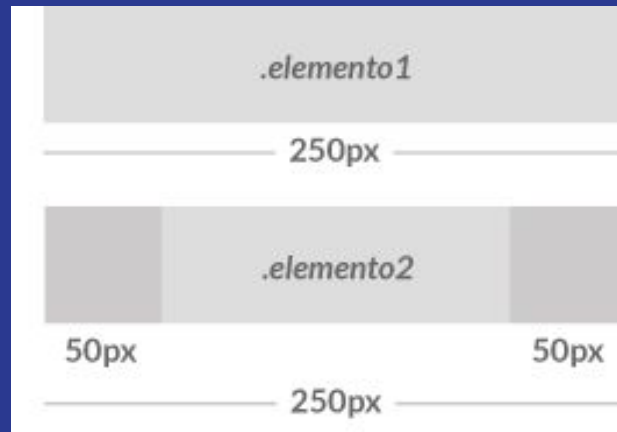
```
1. box-sizing: border-box;
```

O que ela faz? Simples. Ela altera o comportamento do **box-model**, fazendo com que o navegador calcule a largura/altura do elemento contando não apenas o seu conteúdo (como visto no **content-box**), mas também considerando o **padding** (*espaçamento*) e **border** (*borda*) do elemento.

MÓDULO III

O exemplo anterior, com a aplicação do **border-box**:

```
1. .elemento1,  
2. .elemento2 { box-sizing: border-box; }  
3.  
4. .elemento1 {  
5.     background: #ddd;  
6.     width: 250px;  
7.     height: 50px;  
8. }  
9.  
10. .elemento2 {  
11.     background: #ddd;  
12.     width: 250px;  
13.     height: 50px;  
14.     padding: 0 50px;  
15. }
```



MÓDULO III

Propriedade Background Inserindo fundo nos elementos

O background trata do fundo dos elementos. O elemento que mais recebe esta propriedade é o *body*.
As propriedades do background são:

color

Define a cor do fundo.

Possíveis valores:

- **inherit**: Herda o valor do elemento pai.
- **transparent**: Define cor como transparente.
- **<cor>**: Define uma determinada cor.

```
1. div {  
2.   background-color: gray;  
3. }
```

MÓDULO III

background-image

image

Define a imagem de fundo.

Possíveis valores:

- **inherit**: Herda o valor do elemento pai.
- **none**: Indica que não há imagem de fundo.
- **url()**: Define a localização da imagem de fundo.

```
1.  div {  
2.      background-image: url("../img/bg.png");  
3.  }
```

MÓDULO III

background-repeat

repeat

Define a posição de repetição da imagem de fundo.

Possíveis valores:

- **inherit**: Herda o valor do elemento pai.
- **repeat**: Repete a imagem vertical e horizontalmente.
- **repeat-x**: Repete a imagem horizontalmente.
- **repeat-y**: Repete a imagem verticalmente.
- **no-repeat**: Não repete a imagem.

```
1.  div {  
2.    background-repeat: no-repeat;  
3.  }
```

MÓDULO III

background-attachment

attachment

Define se a imagem definida irá rolar juntamente com o viewport, ou se irá ficar fixo.

Possíveis valores:

- **inherit**: Herda o valor do elemento pai.
- **fixed**: Mantém a imagem de fundo fixa na posição definida independente da rolagem do viewport.
- **scroll**: A imagem de fundo acompanha a rolagem do viewport.

```
1.  div {  
2.    background-attachment: fixed;  
3.  }
```

MÓDULO III

background-position

position

Define a posição da imagem de fundo. Observe que esta propriedade necessita de atenção de acordo com o *background-repeat* definido.

Possíveis valores:

- **inherit**: Herda o valor do elemento pai.
- **<porcentagem>**: sendo *A* e *B* dois números, quando declaramos o position com *A%* e *B%*, a posição *A,B* da imagem ficará na posição *A%*, *B%* do elemento.
- **<comprimento>**: sendo *A* e *B* dois números, quando declaramos o position com *Ap*x e *B*p>x, a posição *A,B* da imagem ficará na posição *Ap*x, *B*p>x do elemento.
- **top**: Equivale a 0% na posição vertical.
- **bottom**: Equivale a 100% na posição vertical.
- **left**: Equivale a 0% na posição horizontal.
- **right**: Equivale a 100% na posição horizontal.
- **center**: Caso não tenha sido declarado um valor horizontal, equivale a 50% na posição horizontal, ou, caso não tenha sido declarado um valor vertical, equivale a 50% na posição vertical.

```
1. div {  
2.   background-position: top center;  
3. }
```

MÓDULO III

background

background

É uma propriedade abreviada que une todas as propriedades do background.

```
1.  div {  
2.    background: gray url("../img/bg.png") no-repeat fixed top center;  
3.  }
```


MÓDULO III

Propriedade Font Formatando fonts de texto

A **font** trata das características de todos os textos do site. Ela pode ser aplicada diretamente ao elemento **body** e por consequência todos os elementos filhos herdarão esta propriedade.

As propriedades da **font** são:

family

Determina a família da font a ser usada. Existem dois tipos básicos: as **famílias** e as **famílias genéricas**

As famílias genéricas podem ser três:

- **serif**: Font com serifa.
- **sans-serif**: Font sem serifa.
- **monospace**: Font com a mesma largura em todos os caracteres.

Serifa nada mais é do que aquelas pontinhas que tem em algumas fontes tipo Times, ou seja, Times é uma font com serifa e Arial é uma font sem serifa.

MÓDULO III

```
1.      div {  
2.          font-family: "Times New Roman", Times, serif;;  
3.      }
```

Entre aspas vai o nome da font, seguido da família (Times) e depois da família genérica (serif).

MÓDULO III

Propriedade Overflow

Cuidando dos limites de conteúdo em caixas

A propriedade overflow define o comportamento de um elemento quando suas dimensões são excedidas pelo conteúdo.

Os valores possíveis dessa propriedade são os seguintes:

inherit

Herda a propriedade do elemento pai.

hidden

O conteúdo excedido não ficará visível.

initial

Ajusta a propriedade para o valor padrão.

scroll

O conteúdo excedido será cortado mas haverá uma barra de rolagem para permitir a visualização completa do mesmo.

visible

É o valor padrão e define que o conteúdo não será cortado se exceder o tamanho do elemento.

auto

A barra de rolagem será adicionada automaticamente se o conteúdo exceder o tamanho do elemento.

MÓDULO III

Propriedade Display

Entendendo e manipulando o fluxo dos elementos

Entender a propriedade **display** é fundamental para que possamos compreender o fluxo e estruturação de uma página web. Todos os elementos por padrão já possuem um valor para a propriedade e, geralmente estas são **block** ou **inline**.

Essa propriedade **CSS** especifica o tipo de renderização do elemento na página. Uma definição que pode auxiliar no entendimento da propriedade é a do Chris Coyier, bastante reconhecido no mundo do CSS, que é a seguinte:

*Todo elemento em uma página web é renderizado como uma caixa retangular. A propriedade **display** de CSS vai determinar como essa caixa vai se comportar*

MÓDULO III

Os possíveis tipos

Inline

O elemento se comporta como um elemento em linha.

Exemplos de elemento que se comportam assim são por exemplo as tags **span** e **a**.

None

Ao contrários dos valores atuais, o valor **none** permite, informalmente dizendo, que você *desative a propriedade do elemento*. Quando você utiliza essa propriedade, o elemento e todos seus elementos filhos não são renderizados na página.

Uma coisa importante a ressaltar que a propriedade **display: none** não é a mesma coisa da propriedade **visibility: hidden**. Nessa última o elemento não aparece na tela mas é renderizado na página, ou seja, vemos um *espaço vazio* no lugar do elemento; já a propriedade **display: none** não renderiza o elemento e, o espaço que era ocupado por ele, é ocupado pelo elemento seguinte no fluxo do documento.

MÓDULO III

Block

O elemento se comporta como um bloco. Ocupando praticamente toda a largura disponível na página. Elementos de parágrafo (**p**) e título(**h1**, **h2**, ...) possuem esse comportamento por padrão.

Table

O elemento se comporta como uma tabela.

Inline-block

Semelhante ao **inline**, no entanto, ao definirmos **inline-block** em um elemento, conseguimos definir as propriedades de largura e altura para ele. Coisa que não conseguimos em um elemento com **display: inline**.

MÓDULO III

Propriedade Margin e Padding Manipulando espaços nos elementos

Margin

É a margem do elemento, ou seja, o espaçamento externo do elemento.

Algumas formas de escrita:

margin-top

Define a margem superior do elemento.

```
1.  div {  
2.    margin-top: 10px;  
3.  }
```

margin-right

Define a margem direita do elemento.

```
1.  div {  
2.    margin-right: 20px;  
3.  }
```

MÓDULO III

margin-bottom

Define a margem inferior do elemento.

```
1.  div {  
2.    margin-bottom: 30px;  
3.  }
```

margin-left

Define a margem esquerda do elemento.

```
1.  div {  
2.    margin-left: 40px;  
3.  }
```


MÓDULO III

margin

Define a margem do elemento. Este tipo de declaração pode ser dar de diversas formas.

```
1.  /* t = topo; d = direita; b = baixo; e = esquerda; */
2.  .div1 {
3.      /* t = 20px; d = 20px; b = 20px; e = 20px; */
4.      margin: 20px;
5.  }
6.  .div2 {
7.      /* t = 10px; d = 20px; b = 10px; e = 20px; */
8.      margin: 10px 20px;
9.  }
10. .div3 {
11.     /* t = 10px; d = 20px; b = 30px; e = 20px; */
12.     margin: 10px 20px 30px;
13. }
14. .div4 {
15.     /* t = 10px; d = 20px; b = 30px; e = 40px; */
16.     margin: 10px 20px 30px 40px;
17. }
```

MÓDULO III

Padding

É o preenchimento do elemento, ou seja, o espaçamento interno do elemento.
Algumas formas de escrita:

padding-top

Define o preenchimento superior do elemento.

```
1.  div {  
2.    padding-top: 10px;  
3.  }
```

padding-right

Define o preenchimento direito do elemento.

```
1.  div {  
2.    padding-right: 20px;  
3.  }
```

MÓDULO III

padding-bottom

Define o preenchimento inferior do elemento.

```
1.  div {  
2.    padding-bottom: 30px;  
3.  }
```

padding-left

Define o preenchimento esquerdo do elemento.

```
1.  div {  
2.    padding-left: 40px;  
3.  }
```

MÓDULO III

padding

Define o preenchimento geral do elemento. Assim como na margem, o padding pode declarado de várias formas.

```
1.  /* t = topo; d = direita; b = baixo; e = esquerda; */
2.  .div1 {
3.      /* t = 20px; d = 20px; b = 20px; e = 20px; */
4.      padding: 20px;
5.  }
6.  .div2 {
7.      /* t = 10px; d = 20px; b = 10px; e = 20px; */
8.      padding: 10px 20px;
9.  }
10. .div3 {
11.     /* t = 10px; d = 20px; b = 30px; e = 20px; */
12.     padding: 10px 20px 30px;
13. }
14. .div4 {
15.     /* t = 10px; d = 20px; b = 30px; e = 40px; */
16.     padding: 10px 20px 30px 40px;
17. }
```

MÓDULO III

Propriedade Position

Posicionando elementos via coordenadas

Determina a posição do elemento.

Possíveis valores:

relative

A posição do elemento é relativa ao elemento anterior.

```
1. aside {  
2.     position: relative;  
3. }
```

MÓDULO III

fixed

A posição do elemento é relativa ao viewport. Esta posição não é influenciada pela rolagem da página.

```
1. aside {  
2.     position: fixed;  
3. }
```

absolute

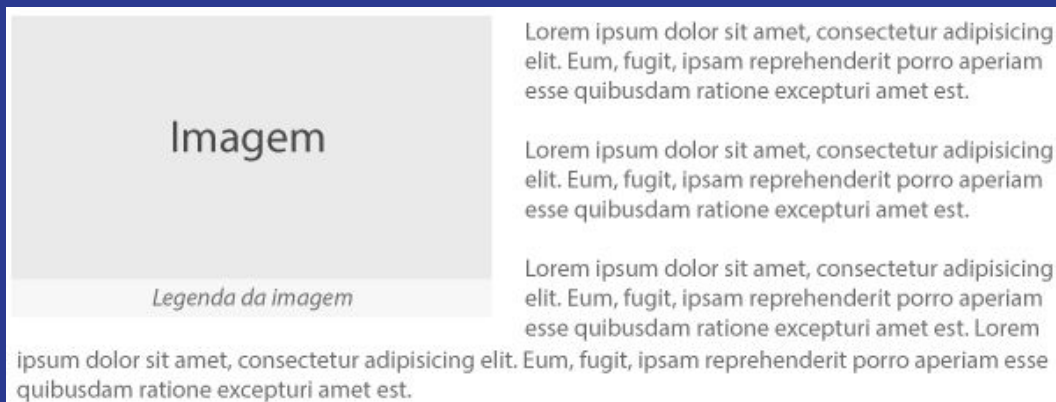
A posição do elemento é relativa ao viewport, ou ao elemento pai quando este tem um *position* definido. Esta posição é influenciada pela rolagem da página.

```
1. aside {  
2.     position: absolute;  
3. }
```

MÓDULO III

Propriedade Float e Clear Estruturando e flutuando elementos

Algo comum ao lermos alguma matéria em revistas ou jornais é uma imagem relacionada ao texto estar posicionada de um lado e, o conteúdo textual contornar a imagem. Algo parecido com isso:



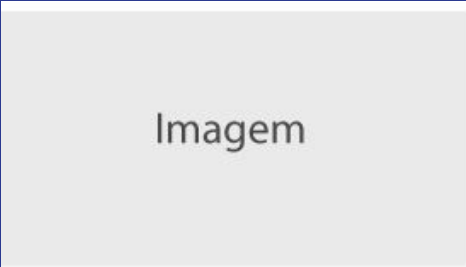
MÓDULO III

E se precisássemos desenvolver um *template* parecido com a imagem.
Provavelmente começaríamos com um código HTML assim:

```
<div>
  <figure>
    
    <figcaption>Legenda da imagem"</figcaption>
  </figure>
  <p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. Eum, fugit, ipsam reprehenderit porro aperiam esse quibusdam ratione excepturi amet est.</p>
  <p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. Eum, fugit, ipsam reprehenderit porro aperiam esse quibusdam ratione excepturi amet est.</p>
  <p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. Eum, fugit, ipsam reprehenderit porro aperiam esse quibusdam ratione excepturi amet est.
    Lorem ipsum dolor sit amet, consectetur adipisicing elit. Eum, fugit, ipsam reprehenderit porro aperiam esse quibusdam ratione excepturi amet est</p>
</div>
```


MÓDULO III

O que teríamos seria algo assim:



Imagem

Legenda da imagem

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Eum, fugit, ipsam reprehenderit porro aperiam esse quibusdam ratione excepturi amet est.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Eum, fugit, ipsam reprehenderit porro aperiam esse quibusdam ratione excepturi amet est.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Eum, fugit, ipsam reprehenderit porro aperiam esse quibusdam ratione excepturi amet est. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Eum, fugit, ipsam reprehenderit porro aperiam esse quibusdam ratione excepturi amet est. ipsum dolor sit amet, consectetur adipiscing elit. Eum, fugit, ipsam reprehenderit porro aperiam esse quibusdam ratione excepturi amet est.

MÓDULO III

Por que isso? O que acontece é que a tag **figure** ocupa toda a largura da página e entra no empilhamento do documento/página, o que não permite que outros elementos apareçam ao seu lado.

É aí que entra o ***float***. Em resumo, essa propriedade permite que tiremos um elemento do fluxo vertical da página e automaticamente faz com que o conteúdo que venha a seguir *flutue* ao seu redor.

O **CSS** ficaria assim:

```
1. figure {  
2.     float: left;  
3.     margin: 0 10px 10px 0;  
4. }
```

MÓDULO III

O que fizemos foi *flutuar* a tag **figure** para o lado esquerdo e, colocar uma margem inferior e direita para que os parágrafos não fiquem grudados na imagem. Com isso conseguiríamos reproduzir um layout igual ao da primeira imagem. Poderíamos até variar e colocar nossa imagem do lado direito:

```
1. figure {  
2.     float: right;  
3.     margin: 0 0 10px 10px;  
4. }
```

MÓDULO III

Aqui mudamos a direção que a tag **figure** vai flutuar e alteramos a margem do lado direito para o lado esquerdo. E teríamos um resultado assim:

Lorem ipsum dolor sit amet, consectetur adipisicing elit. Eum, fugit, ipsam reprehenderit porro aperiam esse quibusdam ratione excepturi amet est.

Lorem ipsum dolor sit amet, consectetur adipisicing elit. Eum, fugit, ipsam reprehenderit porro aperiam esse quibusdam ratione excepturi amet est.

Lorem ipsum dolor sit amet, consectetur adipisicing elit. Eum, fugit, ipsam reprehenderit porro aperiam esse quibusdam ratione excepturi amet est. Lorem ipsum dolor sit amet, consectetur adipisicing elit. Eum, fugit, ipsam reprehenderit porro aperiam esse quibusdam ratione excepturi amet est.

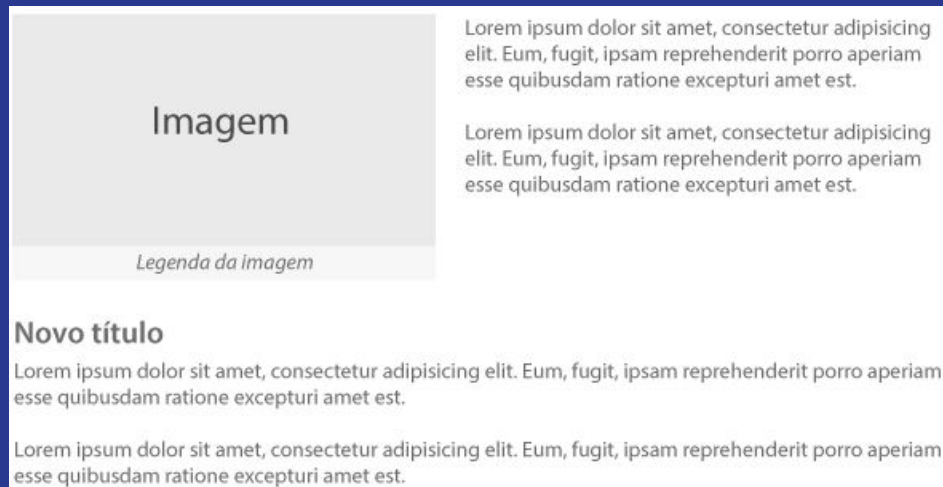
Imagem

Legenda da imagem

MÓDULO III

A propriedade *clear*

Continuando com o nosso exemplo anterior, imagine que agora precisássemos inserir um novo título com mais parágrafos abaixo da imagem. Algo parecido com isso: (para fins de exemplo, diminui o conteúdo textual ao redor da imagem)



MÓDULO III

Para representar esse conteúdo, teríamos um *HTML* semelhante a esse:

```
<div>
  <figure>
    
    <figcaption>Legenda da imagem"</figcaption>
  </figure>
  <p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. Eum, fugit, ipsam reprehenderit porro aperiam esse quibusdam ratione excepturi amet est.</p>
  <p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. Eum, fugit, ipsam reprehenderit porro aperiam esse quibusdam ratione excepturi amet est.</p>
  <h3>Novo título</h3>
  <p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. Eum, fugit, ipsam reprehenderit porro aperiam esse quibusdam ratione excepturi amet est.</p>
  <p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. Eum, fugit, ipsam reprehenderit porro aperiam esse quibusdam ratione excepturi amet est.</p>
</div>
```

MÓDULO III

No entanto, chegaríamos num resultado assim:

Imagem <i>Legenda da imagem</i>	Lorem ipsum dolor sit amet, consectetur adipisicing elit. Eum, fugit, ipsam reprehenderit porro aperiam esse quibusdam ratione excepturi amet est. Lorem ipsum dolor sit amet, consectetur adipisicing elit. Eum, fugit, ipsam reprehenderit porro aperiam esse quibusdam ratione excepturi amet est. Novo título Lorem ipsum dolor sit amet, consectetur adipisicing elit. Eum, fugit, ipsam reprehenderit porro aperiam esse quibusdam ratione excepturi amet est.
esse quibusdam ratione excepturi amet est. Lorem ipsum dolor sit amet, consectetur adipisicing elit. Eum, fugit, ipsam reprehenderit porro aperiam esse quibusdam ratione excepturi amet est.	

MÓDULO III

Por que isso? Como definimos que a tag **figure** flutuaria à esquerda, saindo assim do fluxo vertical da página, todo conteúdo que vem após ela começa a preencher o espaço ao redor da imagem. O que acontece é que os parágrafos que vem logo após a tag **figure** são menores que a altura da imagem, fazendo com que o título (tag **h3**) ao invés de ser renderizada abaixo da imagem, apareça ao lado dela e seguindo o fluxo do documento.

É aí que entra a propriedade **clear**. Ela tem a função de controlar o comportamento de elementos que aparecem no fluxo do documento após determinado elemento que possui a propriedade **float**. Em outras palavras, ela especifica se um elemento deve ser posicionado ao lado de elementos com **float** ou se devem ser colocados abaixo deles. A propriedade aceita 4 valores:

MÓDULO III

- **left:** Elemento é empurrado para baixo de elementos com *float left*;
- **right:** Elemento é empurrado para baixo de elementos com *float right*;
- **both:** Elemento é empurrado para baixo de elementos com *float left* ou *right*;
- **none:** Elemento não é empurrado para baixo de elementos com *float*.

O CSS ficaria assim:

```
1. figure {  
2.     float: left;  
3.     margin: 0 10px 10px 0;  
4. }  
5.  
6. h3 { clear: both; }  
7. /* Pelo fato da tag figure utilizar float left, aqui também poderíamos usar clear: left; */
```

MÓDULO III

E com isso, conseguiríamos fazer com que o título (tag **h3**) fosse empurrado para baixo, sendo renderizado abaixo da tag **figure** e ficando igual ao layout que queríamos.

Mais exemplos

Esse foi apenas um exemplo da utilização da propriedade *float*. Outros casos que você pode ver em vários websites são:

Menus de navegação

Float usado nos itens da lista. Mais precisamente em cada `li`, para dispor os itens do menu na horizontal.

Grids

Float usado para dividir partes do grid. Por exemplo, colocar um bloco de conteúdo à esquerda e um outro bloco à direita.

MÓDULO III

EXERCÍCIOS...

MÓDULO III

FIM...